EXPERIMENT 08

Stochastic Part-of-Speech Tagging using Probabilistic Model

AIM

To implement a stochastic Part-of-Speech tagging algorithm using emission and transition probabilities (Hidden Markov Model / Unigram-Bigram model).

PROCEDURE

1. Collect an annotated training corpus of (word, tag) pairs.
2. Calculate Emission Probabilities $P(w_i | t_i) = \frac{Count(w_i, t_i)}{Count(t_i)}$.
3. Calculate Transition Probabilities $P(t_i | t_{i-1}) = \frac{Count(t_{i-1}, t_i)}{Count(t_{i-1})}$.
4. Implement a maximum-likelihood tagging function to assign the most probable tag sequence for given test sentences.
5. Output the tokens and their stochastically assigned POS tags.

PROGRAM

```
from collections import defaultdict, Counter

training_corpus = [
    [("the", "DET"), ("cat", "NOUN"), ("saw", "VERB"), ("a", "DET"), ("dog",
"NOUN")],
    [("a", "DET"), ("dog", "NOUN"), ("barked", "VERB"), ("at", "ADP"), ("the",
"DET"), ("cat", "NOUN")],
    [("the", "DET"), ("man", "NOUN"), ("saw", "VERB"), ("the", "DET"), ("park",
"NOUN")],
    [("a", "DET"), ("fast", "ADJ"), ("cat", "NOUN"), ("runs", "VERB")]
]

tag_counts = Counter()
transitions = defaultdict(Counter)
emissions = defaultdict(Counter)

for sentence in training_corpus:
    prev_tag = "<s>"
    tag_counts[prev_tag] += 1
    for word, tag in sentence:
        tag_counts[tag] += 1
        transitions[prev_tag][tag] += 1
        emissions[tag][word.lower()] += 1
        prev_tag = tag

def stochastic_tagger(tokens):
    assigned_tags = []
    prev_tag = "<s>"
    for word in tokens:
```

```
        w = word.lower()
        best_tag = "NOUN"
        max_prob = -1.0

        for candidate_tag in ["NOUN", "VERB", "DET", "ADJ", "ADP"]:
            t_count = transitions[prev_tag][candidate_tag]
            p_trans = (t_count + 1) / (tag_counts[prev_tag] + len(tag_counts))

            e_count = emissions[candidate_tag][w]
            p_emit = (e_count + 1) / (tag_counts[candidate_tag] + 100)

            prob = p_trans * p_emit
            if prob > max_prob:
                max_prob = prob
                best_tag = candidate_tag

        assigned_tags.append((word, best_tag))
        prev_tag = best_tag
    return assigned_tags

test_sentence = ["The", "dog", "saw", "a", "cat"]
results = stochastic_tagger(test_sentence)

print("--- Stochastic POS Tagging Output ---")
for word, tag in results:
    print(f"Token: {word:<8} ---> Tag: {tag}")
```

OUTPUT

```
> python3 experiment_08.py
--- Stochastic POS Tagging Output ---
Token: The      ---> Tag: DET
Token: dog      ---> Tag: NOUN
Token: saw      ---> Tag: VERB
Token: a        ---> Tag: DET
Token: cat      ---> Tag: NOUN
[Process completed with exit code 0]
```

RESULT

Thus, the stochastic part-of-speech tagging algorithm based on probabilistic transitions and emissions was successfully implemented.